

Impact of “Evaluating Software Complexity Measures”

Elaine J. Weyuker 

Abstract—We investigate the impact that the paper “Evaluating Software Complexity Measures” has had over the thirty-six year period since it was first published in 1988. We look both at how the frequency of citation increased, and the ways that the research has been used. We describe how citations evolved from the initial obvious intent of assessing newly proposed complexity measures to assessing qualities other than complexity, to using them to assess complexity in entities other than software code, or restricted types of programming paradigms, to the adoption of our approach or properties in fields distinct from computing.

Index Terms—Software engineering, software metrics, software measurement.

I. BACKGROUND

IN December 1986 I published a paper “Axiomatizing Software Test Data Adequacy” [1] in which a general axiomatic theory of test data adequacy was proposed, and then used to evaluate five well-known notions of adequacy which were frequently cited in the literature. The underlying motivation for this research was that adequacy criteria are used to determine whether or not it is safe to stop testing because sufficient testing has been performed to assure that the software will behave acceptably with high confidence. Therefore, we should have agreed-upon criteria for making such a decision. For that reason, by establishing this type of theoretical axiomatic framework, potential adequacy criteria users should be able to identify the strengths and weaknesses of a proposed criterion.

We showed that the set of eleven axioms we proposed were consistent because two of the five adequacy criteria considered satisfied all of the axioms, while three of the criteria failed to satisfy several of the axioms and therefore might be considered poor choices to use as determinants of whether or not it was safe to stop testing. A motivating argument was offered for each of the axioms, and examples were offered showing the consequences of failing to satisfy each axiom. Our goal was to encourage other researchers to identify other essential characteristics of adequacy criteria and propose additional axioms. With a good comprehensive set of axioms, the software testing

community could eventually use them to assess newly proposed adequacy criteria or even use them to steer the development of new criteria.

Much to my surprise, while many mathematically trained researchers found the work compelling, some Software Engineering researchers appeared unhappy with the use of the word *axiom* itself. This led to a number of “meta papers” in the sense that rather than using the axioms in ways we had envisioned, they instead argued about the appropriateness of the entire enterprise. These types of complaints became a major distraction. Instead of focusing on the point of the research, namely that we needed to have a commonly agreed-upon and concrete definition of what constituted a good adequacy criterion, for some researchers, the focus was on totally extraneous issues. While the paper has been cited close to 300 times, it has not had the impact I had expected it would. I was particularly eager for the work to have an influence on the practitioner community, which sadly continued using woefully inappropriate adequacy criteria such as: we stop when we’ve run out of time and money, or we stop testing when all of the test cases have run correctly. While the latter may sound like a reasonable stopping criterion, it is in fact largely useless since it rewards both small and non-challenging test sets.

As someone trained in the theory of computation, and the coauthor of a widely used text on the subject [2], I felt that too much of Software Engineering research consisted of papers that had the following high level framework: Here’s a new way to do something, and I’ve thought it up and I’m a smart guy so it must be good. If the newly proposed technique was illustrated at all, it was typically illustrated on very small, toy, programs. I felt strongly that we needed to move the field so that rigor became the norm.

This idea led to my investigating ways to formalize software complexity measures in much the same way I had tried to formalize software test data adequacy measures. However, I decided to use the word *property* rather than *axiom* as a keyword, so that other researchers might find it less intimidating and might contribute to my initial set of so-called properties. I presented a carefully defined set of nine properties for deciding whether or not a proposed software complexity measure was acceptable. I then considered four previously proposed and frequently referenced complexity measures to see which of the properties each satisfied, why they failed to satisfy certain properties, whether the failure to satisfy one or more of the properties meant the criterion was unusable, and if so, how it could be rectified. The result of this investigation was published

Received 13 January 2025; accepted 19 January 2025. Date of publication 24 January 2025; date of current version 17 March 2025. Recommended for acceptance by S. Uchitel.

The author is with the University of Central Florida, Orlando, FL 32816 USA (e-mail: weyuker@gmail.com).

Digital Object Identifier 10.1109/TSE.2025.3533467

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

in *IEEE Transactions on Software Engineering* in September 1988 with the title *Evaluating software complexity measures*.

Unlike the adequacy criterion paper, this paper was largely warmly embraced by the research community from the outset and has been regularly cited during the more than 35 years since it was originally published. Of course I will never know whether the reason the software complexity property paper was cited more than four and a half times as frequently as the test data adequacy axiom paper is due to the fact that many more people find software complexity a more compelling research topic than test data adequacy. However, I strongly suspect that part of the reason for the much higher level of impact of the software complexity paper as compared to the test data adequacy paper is due, at least in part, to my avoidance of the word *axiom*.

II. IMPACT

We now consider the impact of our paper from several perspectives including the frequency of citation over the years and the nature of the citations. We only mention a sampling of types of uses of our properties to give a flavor of how the impact evolved and only include specific references to these citing papers in cases where our paper played some sort of central role in the citing paper's research. Further, we only cite individual papers if they are the first of a type or really unusual in the sense that they have used our axiomatic framework or approach in a totally new area or field. Generally we just describe a variety of types of roles our paper has played, and how those roles evolved with time.

We therefore focus on the impact of this research in terms of frequency of citations from papers in the following broad categories. The first type of citing paper either discusses or extends the set of proposed properties themselves. We consider these papers methodological. We also include papers that propose new complexity measures, either for all programs or for restricted classes of programs, and use our framework to assess the new measures. Another category we include consists of papers that investigate how to use or adapt our approach to artifacts other than software. All of these types of papers are considered central and intended uses of the axiomatic framework outlined in our paper, and we consider all of these types of papers important when considering our paper's impact.

When discussing the frequency of citation, we have relied on Google Scholar to obtain that data. We acknowledge that there are other possible sources of this data including ResearchGate, and have noted that there are cases of citations identified by each and not identified by the other, but have found the Google Scholar data to be more comprehensive. Another thing we've noticed is that it sometimes takes a significant amount of time for a citation to appear in the Google Scholar data and so, especially for the most recent period we consider, the Google Scholar data may be a significant underestimation of the number of citations for a given time period, both because of the delay in the identification by Google Scholar and the possibility that it never is actually included in the data. Nonetheless, the number of citations of this paper listed in Google Scholar (1350 as of the

end of 2024) is a good indication of how important or impactful a publication is.

Beyond the initial two year period, we considered citations during five year intervals to see how the frequency and nature of citations changed with time. The reason for isolating the first two years was a pragmatic one based on the typical time it took for a paper from submission to publication in archival Software Engineering journals such as the *IEEE Transactions on Software Engineering* where the paper was published. For example, while this paper was submitted in February 1986, and quickly accepted with minor revisions which were submitted in May 1986, the paper did not appear until the September 1988 volume of the journal. This two and a half year lag from submission to publication was an entirely typical interval for a paper that received favorable reviews, needed little revision which was done quickly, and was ultimately published. Note that at the time, there were no online editions published prior to the print issue being available.

Therefore we were not surprised when there were only three citations listed for the 1988 and 1989 time period. Of these citations, one appeared in a thesis, one was in a conference proceedings and cited our paper in an inconsequential way, and the last one was in an invited article that I authored in a special issue of the Communications of the ACM which was published in June 1988, prior to the appearance of this paper. The citation was listed as *to be published*.

III. 1990–1994

The first five year period that we considered, 1990–1994, had 80 citations according to Google Scholar. As we had hoped, many of the citations during this period were from researchers defining new complexity criteria and using our properties to assess their strengths and potential weaknesses or to critique newly proposed complexity measures defined by other researchers. The other major category of citing paper discussed the application of our properties for a specific type of software development paradigm.

One particular paper, co-authored by Chidamber and Kemerer [4], stands out during that period. In this frequently-cited paper, the authors focused on the then relatively newly introduced approach to software development known as Object Orientation [5] because OO Design was widely viewed as a way of improving the process of developing software and was being commonly adopted in practice. Chidamber and Kemerer felt it was therefore especially important to have good software metrics since many organizations were adopting the new technology and standard practices had not yet been established. Therefore, their goal was to develop and implement a new suite of metrics expressly designed for Object Oriented Design. The suite consisted of six design metrics which they assessed using our properties. They found that all of their metrics satisfied most of the properties, but that one specific property was satisfied by none of their metrics. They then presented a persuasive argument that certain characteristics specific to Object Oriented Design made that a reasonable and non-fatal condition. This was another way we had envisioned our properties being used:

to identify characteristics of development paradigms for which a subset or superset of the properties were most appropriate. Thus the properties could also be used as a jumping off point for the evaluation of proposed metrics in new areas.

IV. 1995–1999

According to Google Scholar there were 141 citations of our paper during this five year period, indicating a significant increase in interest in and acceptance of this paper.

The papers published during the second early period were, for the most part, of one of three types. As in the earlier period, the first category included papers providing justifications of newly proposed software complexity measures, with increasing numbers of the measures devised specifically for software developed using the Object Oriented Design paradigm. This was likely due to steadily growing interest in both the research and the practitioner communities in this type of development.

The second category of paper involved the investigation of extending the set of properties to measure complexity in areas in addition to assessing the complexity of static software systems. Lee, for example, as part of her Ph.D. dissertation [6] considered the properties as part of a study of change impact analysis. She argued that this was important because many large industrial systems remain in active use for many years after they are initially deployed, and are therefore regularly modified and extended.

The third category involved emulating the process introduced in our paper for an entirely different concept in the sense of developing a set of properties that any good measure of the relevant concept should possess. In this case, rather than using or modifying the properties we had proposed, the authors instead adopted the method and proposed an entirely different set of properties to assess an entirely different type of measure. For example, inspired by our development of properties that software complexity measures should possess, the authors of [7] developed a set of properties which they claimed any measure that was used to assess the benefit of reusing code should possess and then empirically evaluated them.

Other authors that cited the work during this period considered such areas as security measures, coupling and cohesion, or used formalisms other than axiomatic ones to assess complexity measures. This was another modality we had initially envisioned, since our ultimate goal was to encourage researchers to use more formal methods when introducing new types of software engineering measures or processes.

V. 2000–2004

The third five year interval again saw an increase in citations, to 187. During this period, citations involving Object Oriented Development measures continued to be an important focus, but there was also an increasing focus on formalizing measures of other software characteristics as well as formalizing complexity or other characteristics of entities other than software. Yet another direction that was becoming recognized as important was developing other ways of formalizing measurement.

During this period, papers that cited ours focused on evaluating such disparate issues as the complexity of the Unified Modeling Language (UML) and using complexity metrics to predict maintenance performance using object-oriented design metrics. Other papers used our approach to measure coupling and cohesion, defined metrics for component-based software development, and developed an axiomatic theory for metrics based on distance. Since data warehouses were becoming widely used in practice during this timeframe, there was a formal proposal of metrics for a multidimensional model to assure quality data warehouses. Thus we saw increasingly a recognition of the need for formality when defining metrics in a number of different arenas.

VI. 2005–2009

According to Google Scholar, 248 papers cited this paper during the next five year period, yet another major increase over the previously time period.

During this period there was an even further branching out of domains that found our approach and even our properties themselves useful for assessing some measures. This included the development of quality metrics for business process models, design time complexity measures to decide, pre-implementation, the difficulty of producing various types of processes such as Web, business or workflow, and allowed authors to discard from consideration metrics to identify change-prone classes that do not satisfy one or more of our properties. Other papers used our properties directly to evaluate newly defined metrics designed explicitly for component-based development and integration and for distributed workflow design.

There was also a continuing interest in metrics for Object Oriented Design, as well as a number of papers that cited our paper as a way of formally validating the metrics they are proposing and list it as something to be done in the future.

VII. 2010–2014

This period again saw a noticeable increase in the number of citations over the previous period, listed as 291 citations by Google Scholar.

Papers during this period investigated such things as using an axiomatic theory similar to ours to assess the maintainability of product line feature models, and another defining a set of ontology metrics to measure the design complexity of ontologies and then using our properties to evaluate these new metrics. Another interesting direct application of our properties appeared in a paper in which a quantitative measure for structural complexity was defined and applied to actual engineered systems including a gas turbine engine [8]. The authors showed that their proposed topological metric satisfied all nine of our properties, which incidentally showed that the properties were consistent. One of the authors, Sinha, extended these ideas in his Ph.D. dissertation in the Engineering Systems Division. He used our properties and approach to develop a framework for assessing structural complexity of engineered systems, adding a set of four additional properties to our initial set [9]. Using this

framework, he developed a complexity-based design paradigm for interconnected, complex engineered systems.

VIII. 2015–2019

For the first time since the paper was published, we saw a decrease in the number of citations compared to the previous period, to 232 according to Google Scholar. Nonetheless, given that the paper was published some 30 years earlier, that still indicates substantial interest and impact of the paper.

In a paper describing research focused on building a fault prediction tool [10], the authors first identify the use of several software metrics to build their prediction model. They then use our properties to assess several of their proposed cost metrics. Another interesting use [11] was to formally assess a suite of cognitive complexity metrics the authors created to be used for Object Oriented project evaluation. Yet another paper published during this period argued that for complex engineering systems which were highly modular in design, it was desirable for the variation of complexity of the modules to be minimized [12]. The authors looked at a number of different potential complexity measures and assessed each of them using our properties and based on these assessments selected a measure to use in their empirical study.

IX. 2020–2024

During this most recent period, there were 153 citations according to Google Scholar.

It is particularly interesting to see how the paper has impacted fields far removed from Software Engineering and the assessment of software complexity in isolation. In a recent paper [13] published in the *Journal of Mechanical Design*, which is a journal published by ASME, the American Society of Mechanical Engineers, the authors are concerned about cyber-physical systems and the role of complexity in the behavior of such systems. In particular, they are concerned with how complexity leads to adverse project outcomes and mention specifically cost-overruns and delayed schedules. They state that while a number of complexity measures have been proposed for cyber-physical systems, and the literature generally recognizes that such factors as how interconnected the system is or how increases in size may have an impact on the complexity, there has been little interest in evaluating how complexity should be measured. They selected six complexity measures and created a benchmark set of system architectures which vary across each of the dimensions of concern to see how effective each of their proposed complexity measures were at capturing the increases in complexity. They stated that they were inspired to undertake this work based on our paper.

Another recent paper [14] investigates whether and to what extent our properties are useful in assessing complexity measures for process models rather than software. This is yet another type of usage of our framework: namely using it to assess different, but related objects, in this case process models rather than software code. The authors found that six of our nine properties were important for assessing process models, while three were less relevant but could nonetheless provide interesting

insights that might help one define good complexity measures in their milieu. Thus the authors are reaffirming our position that having formally defined properties is important to assess complexity measures in a discipline different from the one for which we initially defined them. They furthermore found that most of our properties were directly useable for process model assessment. They then augmented our set of properties so that the extended set was particularly relevant for their environment. Again this represents another way that this paper has impacted the world of software development.

X. CONCLUSION

When we reflect on the frequency and ways in which this paper has been cited over the past 35 years since it was first published, we observe that perhaps its biggest impact has been to raise awareness of the lack of formal underpinnings in so much of the early literature on software engineering in general and complexity assessment in particular. This led to the recognition of the importance of having a formal framework for complexity and other software engineering measures. With time, the application of our methodological approach has been widely accepted, even when some of the properties were inappropriate for the particular characteristic being measured or the class of objects under consideration. Initially, most papers that cited this paper either used it to justify the introduction of a new complexity measure or used the set of properties to assess complexity in restricted classes of software systems, specifically those developed using Object Oriented Design. However, with time, the approach has been extended to develop formal frameworks to assess measures of different system characteristics and in different fields such as cyber-physical systems or using different artifacts such as process models.

ACKNOWLEDGMENT

I am grateful to Tom Ostrand for his careful reading of this paper and his suggestions.

REFERENCES

- [1] E. J. Weyuker, “Axiomatizing software test data adequacy,” *IEEE Trans. Softw. Eng.*, vol. SE-12, no. 12, pp. 1128–1138, Dec. 1986.
- [2] M. Davis, R. Sigal, E. J. Weyuker, *Computability, Complexity, and Languages: Fundamentals of Theoretical Computer Science*, 2nd ed. San Diego, CA, USA: Academic, 1994.
- [3] E. J. Weyuker, “Evaluating software complexity measures,” *IEEE Trans. Softw. Eng.*, vol. SE-14, no. 9, pp. 1357–1365, Sep. 1988.
- [4] S. R. Chidamber, and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [5] G. Booch, *Object Oriented Design With Applications*. Redwood City, CA, USA: Benjamin/ Cummings, 1991.
- [6] M.L. Lee, *Change Impact Analysis of Object-Oriented Software*, Ph.D. dissertation, Fairfax, VA, USA: George Mason University, 1998.
- [7] P. Devanbu, S. Karstu, W. Melo and W. Thomas, “Analytical and empirical evaluation of software reuse metrics,” in *Proc. IEEE 18th Int. Conf. Softw. Eng.*, Berlin, Germany, 1996, pp. 189–199.
- [8] K. Sinha and O. L. de Weck, “A network- based structural complexity metric for engineered complex systems,” in *Proc. IEEE Int. Syst. Conf. (SysCon)*, Orlando, FL, USA, 2013, pp. 426–430.

- [9] K. Sinha, *Structural Complexity and Its Implications for Design of Cyber-Physical Systems*, Ph.D. dissertation, Cambridge, MA, USA: Massachusetts Institute of Technology, Feb. 2014.
- [10] L. Kumar, S. K. Sripada, A. Sureka, and S. K. Rath, "Effective fault prediction model developed using least square support vector machine (LSSVM)," *J. Syst. Softw.*, vol. 137, pp. 686–712, Mar. 2018.
- [11] S. Misra, A. Adewumi, L. Fernandez-Sanz, and R. Danasevicius, "A suite of object oriented cognitive complexity metrics," *IEEE Access*, vol. 6, pp. 8782–8796, 2018.
- [12] K. Sinha, and E.S. Suh, "Pareto-optimization of complex system architecture for structural complexity and modularity," *Res. Eng. Des.*, vol. 29, pp. 123–141, Jan. 2018.
- [13] A. Hennig, T. G. Topcu, and Z. Szajnfarder, "So you think your system is complex?: Why and how existing complexity measures rarely agree," *ASME. J. Mech. Des.* vol. 144, no. 4, Apr. 2022.
- [14] P. Schalk, A. Burke and R. Lorenz, "Navigating complexity: Comparing complexity measures with Weyuker's properties," in *Proc. 6th Int. Conf. Process Mining (ICPM)*, Kgs. Lyngby, Denmark, 2024, pp. 145–152.



Elaine J. Weyuker is a University Distinguished Professor, College of Engineering and Computer Science, University of Central Florida. She has also been a Professor with Mälardalen University, Västerås, Sweden, a Fellow and Distinguished Member of the Technical Staff with AT&T Labs and Bell Labs, and a Professor of computer science with the Courant Institute of Mathematical Sciences, New York University. She is a member of the US National Academy of Engineering, and was elected a fellow of the ACM and the IEEE. She received the IEEE's Harlan Mills Award for outstanding software engineering research, and the ACM/SIGSOFT Outstanding Research Award, as well as a number of other awards for her technical excellence and active advocacy of women and underrepresented minorities in computing as well as her extensive volunteer activities. Her research expertise includes techniques and tools to improve the quality of software systems through systematic validation activities, including the development of testing, assessment and software fault prediction models. Another area of research interest has been software metrics and measurement. Prior to that, she did research in *Theory of Computation* and is the co-author of a book *Computability, Complexity, and Languages*.